

Assignment 2: Data Structures

Marat Aruzhan

SE-2525

Design and Analysis of Algorithms

1. Implementation and Method

The project implements three integer data structures: `DynamicArray`, `MyLinkedList`, and `MinHeap`. `DynamicArray` doubles its capacity when full. `MyLinkedList` is a singly linked list with head and tail references. `MinHeap` uses an array to maintain the minimum element at the root. Standard Java collections are used only in tests for comparison.

Experiments were run on Windows 11 (amd64), using OpenJDK 25.0.1. Each benchmark uses five warm-up runs followed by five measured runs, and reports the median execution time. `System.nanoTime()` measures elapsed time. Data preparation and correctness checks are outside the timed section. Input sizes are 100, 1,000, 10,000, and 100,000. Random data uses seed 42 for reproducibility.

Workload	Operations
W1: Indexed access	10,000 random get operations.
W2: Search	1,000 contains queries: 500 successful and 500 unsuccessful.
W3: Insert/remove	1,000 insert/remove pairs at the head and, separately, at the middle.
W4: Min-heap	Insert n values, then extract all n values in sorted order.

W1–W3 use the same shuffled values from 0 to $n-1$ for both list implementations. W3 restores the original size after each insert/remove pair. W4 uses random integer values. Steps, moves, and comparisons are recorded inside the data-structure methods.

2. Complexity Analysis.

Here, n is the current number of elements. Average indexed operations assume uniformly distributed indices. Auxiliary space means additional space used by an operation, excluding existing storage. Amortized bounds describe the cost averaged over a sequence of operations.

Operation	Best	Average / amortized	Worst	Auxiliary space	Reason
Array: append	$O(1)$	$O(1)$ amortized	$O(n)$	$O(n)$	Resizing copies all elements.
Array: add(index, x)	$O(1)$	$O(n)$	$O(n)$	$O(n)$	Shifts elements; may resize.
Array: remove(index)	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Shifts the remaining suffix.
Array: get(index)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Direct array access.
Array: contains(x)	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Sequential search.
List: append	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Uses the tail reference.
List: add(index, x)	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Traverses to the predecessor.
List: remove(index)	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Traverses to the predecessor.
List: get(index)	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Traverses from the head.
List: contains(x)	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Sequential search.
Heap: insert(x)	$O(1)$	$O(\log n)$ amortized	$O(n)$	$O(n)$	Sifts upward; may resize.
Heap: peekMin()	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Reads the root.
Heap: extractMin()	$O(1)$	$O(\log n)$ upper bound	$O(\log n)$	$O(1)$	Sifts downward.

Operation	Best	Average / amortized	Worst	Auxiliary space	Reason
Heap: buildHeap()	$O(n)$	$O(n)$	$O(n)$	$O(n)$	Copies input and uses bottom-up sifting.

Heap insertion without resizing takes $O(\log n)$ in the worst case; a resizing insertion takes $O(n)$. Heap average costs depend on the input distribution. Search averages assume uniformly distributed successful positions or a fixed fraction of unsuccessful queries. Size queries, metric getters, and metric resets take $O(1)$. The array and heap retain $O(\text{capacity})$ storage, while the linked list uses $O(n)$ storage.

3. Loop Invariants

3.1 DynamicArray.add(index, x)

Invariant. Let A be the array contents before shifting and s the original size. Before each iteration with variable i , every position k where $i < k \leq s$ contains $A[k-1]$. Original positions below or equal to i remain unchanged wherever they exist.

Initialization. Initially, $i = s$. The shifted range is empty, and all original elements are unchanged.

Maintenance. The assignment $\text{data}[i] = \text{data}[i-1]$ copies the next element one position to the right. Moving from right to left preserves values that have not yet been copied. Decrementing i maintains the invariant.

Termination. The loop stops when $i = \text{index}$. Positions $\text{index}+1$ through s contain the original suffix shifted right by one position.

Conclusion. Writing x at index and increasing size inserts the new value while preserving the order of all original elements. Any preceding resize preserves the original contents and provides sufficient capacity.

3.2 MyLinkedList.get(index)

Invariant. Before each iteration with variable i , current points to the node at position i .

Initialization. Initially, $i = 0$ and $\text{current} = \text{head}$, which is the node at position 0.

Maintenance. The assignment $\text{current} = \text{current.next}$ advances to position $i+1$. Incrementing i restores the invariant.

Termination. The loop stops when $i = \text{index}$, so current points to the requested node.

Conclusion. Returning current.value returns the element at the requested index. The initial bounds check guarantees that this node exists.

4. Instrumentation and Testing

Counters. Steps count instrumented array reads or linked-list traversals. Moves count element copies, shifts, or structural link updates. Comparisons count value comparisons, excluding index checks and loop conditions. Counters are reset before each measured workload.

Counting conventions. Writing a newly inserted value is excluded from moves in array and heap insertion. Heap `siftDown` counts its final placement as a move, even when the position is unchanged. Local traversal-variable assignments are not structural link updates. These counters describe selected operations, rather than every machine instruction.

Testing. The recorded Maven test run passed all 40 tests: 13 for `DynamicArray`, 13 for `MyLinkedList`, and 14 for `MinHeap`, with no failures or errors. Tests cover empty structures, invalid indices, duplicates, negative values, resizing, insertion, removal, and metric resets. Randomized tests compare list operations with `ArrayList` and heap operations with `PriorityQueue`. Heap tests also check the heap property and sorted extraction order.

5. Experimental Results.

The figures show median execution times and operation counts for each workload. Each point represents a tested input size. Lines connect measured points and do not represent additional measurements. Timing fluctuations may reflect JVM compilation, runtime optimization, and system scheduling.

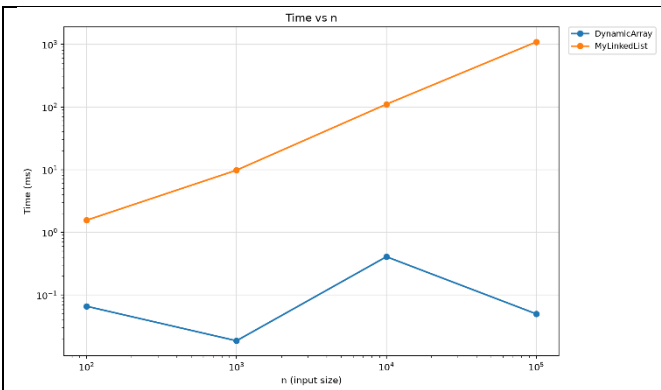


Figure 1. W1: indexed access time.

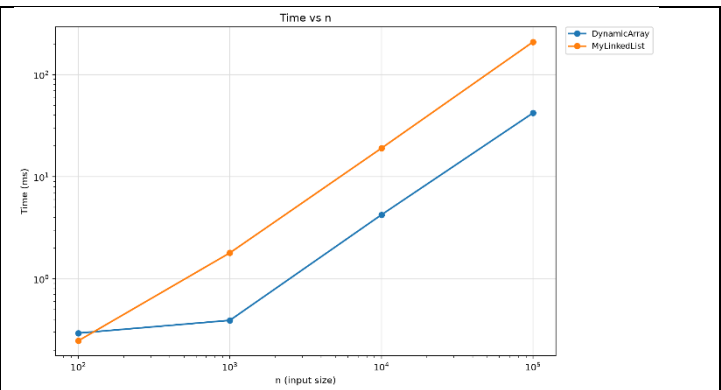


Figure 2. W2: search time.

DynamicArray performs exactly 10,000 indexed reads for every input size. MyLinkedList must traverse nodes from the head and performs 502,489,208 traversal steps at $n = 100,000$. This agrees with $O(1)$ array access and $O(n)$ worst-case linked-list access. Fluctuations in array execution time do not indicate a change in its asymptotic complexity.

W2 performs 76,530,931 value comparisons in each structure at $n = 100,000$. The array takes 42.1269 ms, while the linked list takes 209.1946 ms. Contiguous array storage supports spatial locality, whereas linked-list traversal involves dependent pointer reads and potentially scattered nodes.

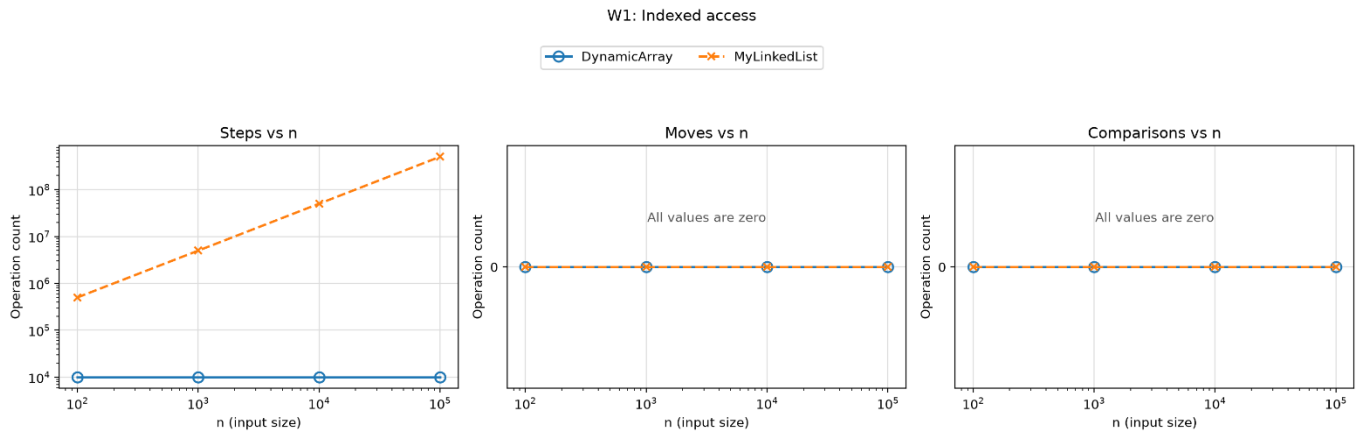


Figure 3. W1: steps, moves, and comparisons.

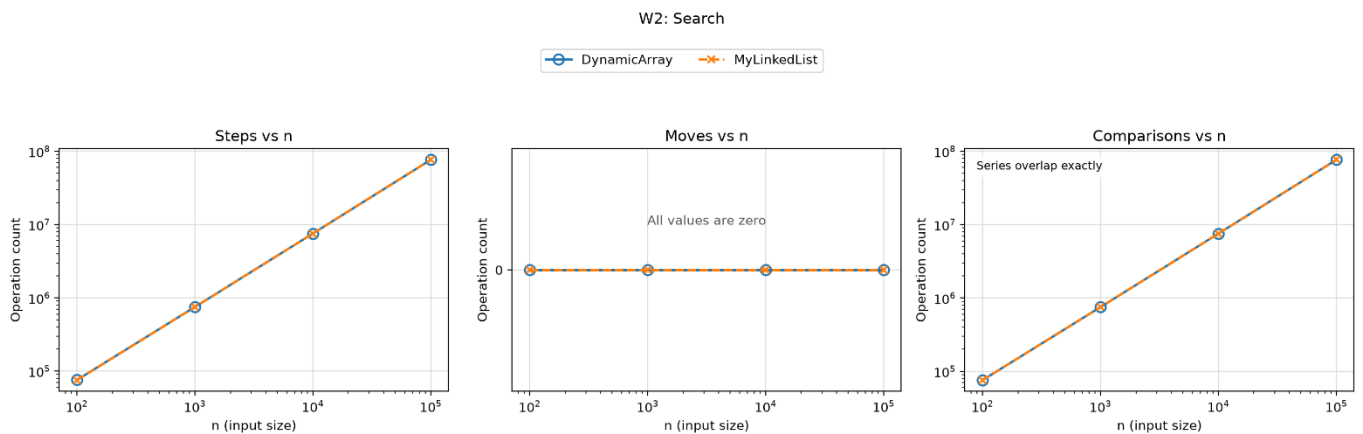
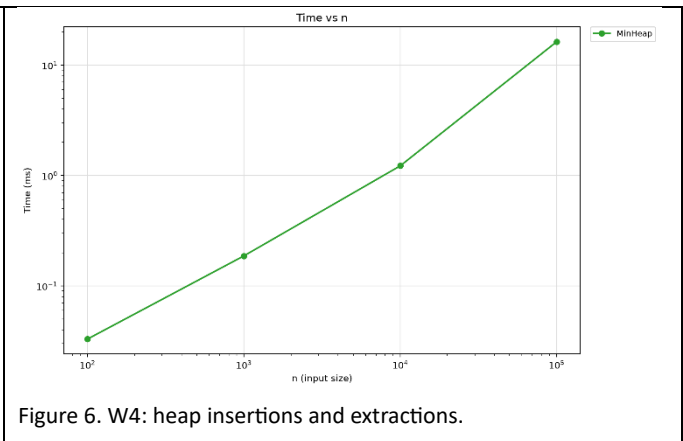
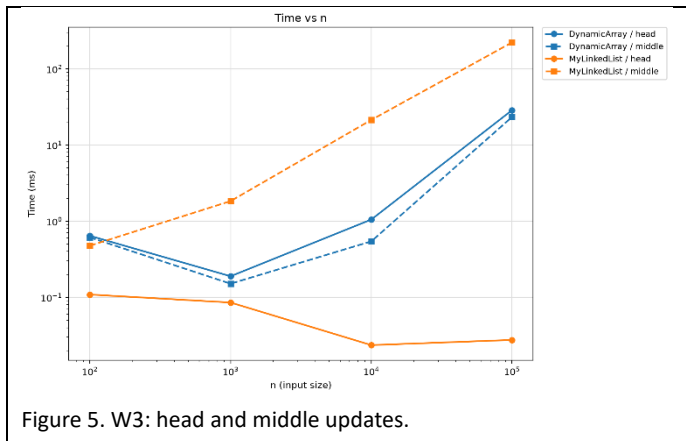


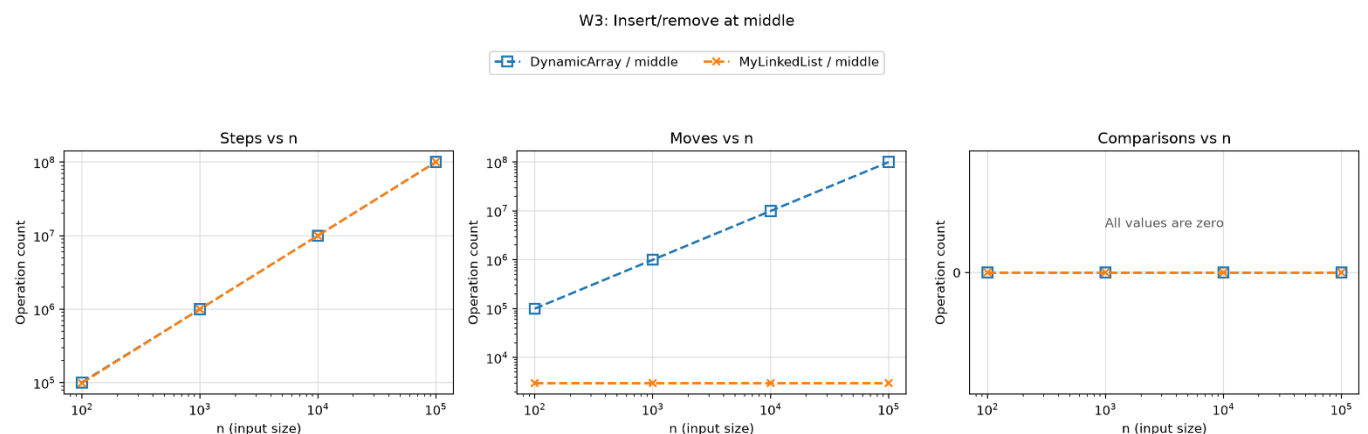
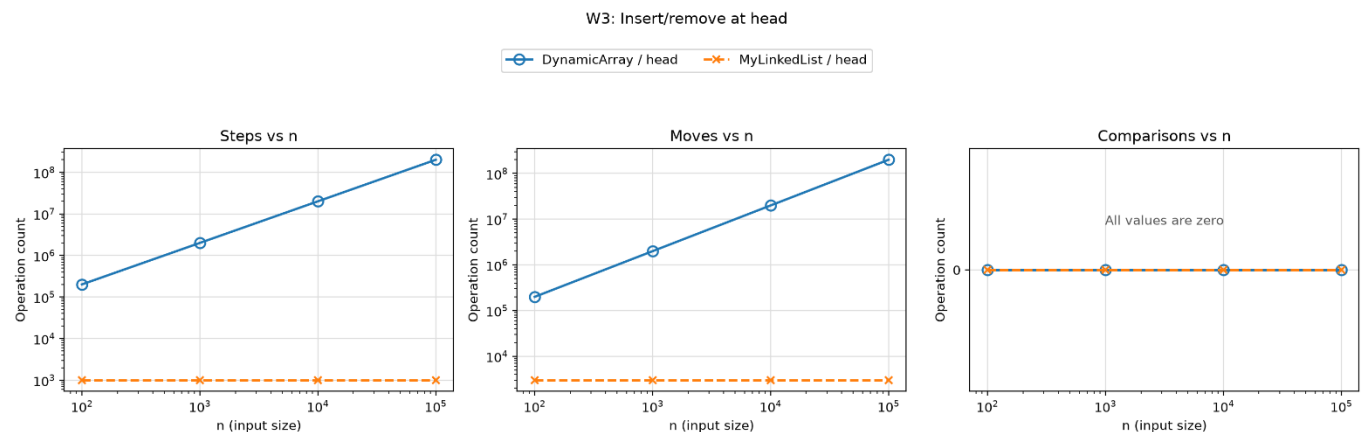
Figure 4. W2: steps, moves, and comparisons.

Neither workload moves elements. W1 performs no value comparisons. In W2, both structures perform the same number of comparisons, so their comparison curves overlap. Linked-list traversal steps are 500 fewer than comparisons because each successful search returns before advancing to the next node.



In W3, linked-list head updates require constant work per operation, while array head updates shift elements. At $n = 100,000$, the array performs 200,000,000 moves for the head workload. Middle updates require linear work in both structures: the array shifts elements, while the list traverses nodes. In the largest middle case, the array takes 23.2990 ms and the list takes 221.8586 ms.

W4 takes 16.2744 ms and performs 3,059,283 value comparisons at $n = 100,000$. Inserting and extracting n elements has an $O(n \log n)$ worst-case total cost; the measurements are consistent with this bound, but do not prove it.



W4: Heap insertions and extractions

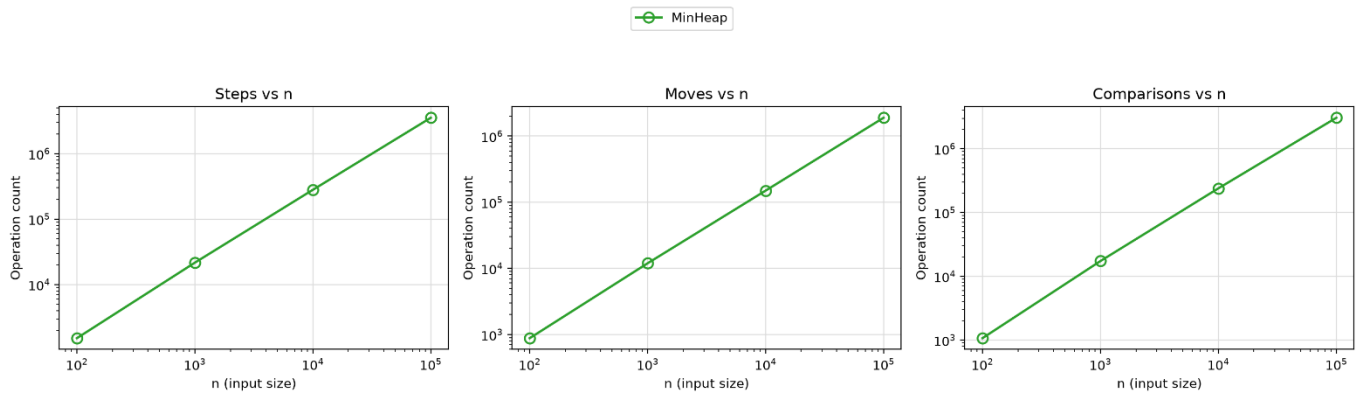


Figure 9. W4: heap operation counts.

6. Bonus: Floyd Heap Construction.

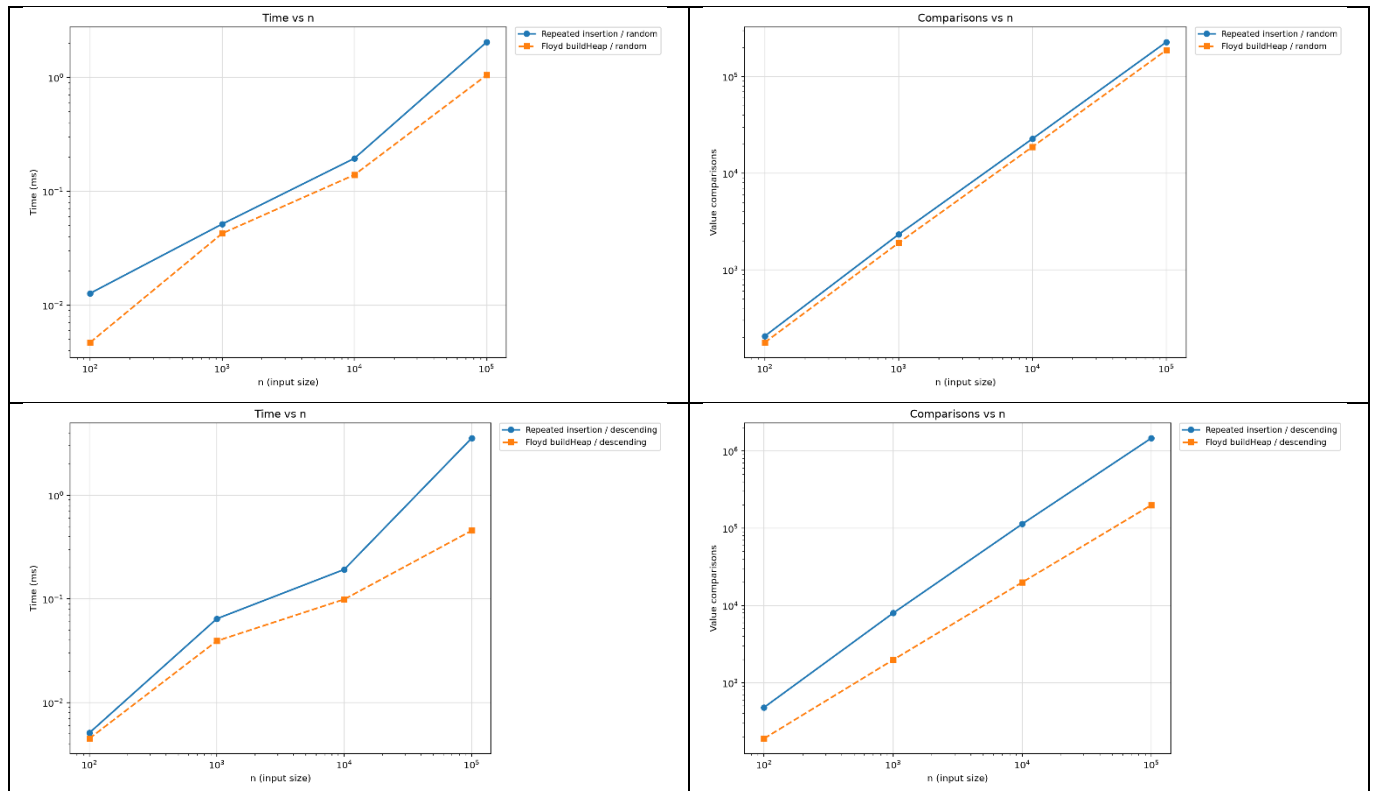


Figure 10. Repeated insertion versus Floyd construction: time and comparisons for random and descending inputs.

Floyd's algorithm builds the heap by sifting internal nodes downward, starting from the last parent. There are $O(n/2^h)$ nodes of height h , so the total work is $O(n \sum h/2^h) = O(n)$. Copying the input also takes $O(n)$. Repeated insertion has $O(n \log n)$ worst-case total cost.

For descending input at $n = 100,000$, repeated insertion performs 1,468,946 comparisons and takes 3.5609 ms. Floyd construction performs 199,978 comparisons and takes 0.4564 ms, approximately 7.8 times faster in this run.

7. Bonus: Memory Footprint.

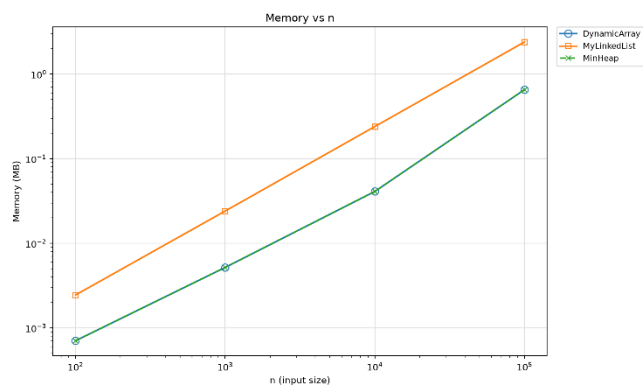


Figure 11. Memory footprint measured with JOL (1 MB = 1,000,000 bytes).

JOL 0.17 measures the total size of objects reachable from each structure. At $n = 100,000$, DynamicArray and MinHeap each occupy 655,424 bytes (0.655424 MB), while MyLinkedList occupies 2,400,048 bytes (2.400048 MB), approximately 3.66 times as much.

In this JVM configuration, each linked-list node occupies 24 bytes, including its header, integer value, reference, and alignment padding. The list contains $n+1$ objects, while each array-based structure contains two. Arrays retain unused capacity after growth. These measurements depend on JVM layout and do not represent total process memory.

8. Conclusion

Arrays suit indexed access and sequential scanning. Linked lists suit frequent head updates, while heaps support repeated minimum extraction. Node allocation can increase garbage-collection pressure, but this experiment does not isolate GC costs. Warm-up runs and medians reduce timing variability without eliminating it.